2026-04-27-perceived-host-suspension-investigation

2026-04-27 — “Perceived host suspension” investigation

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

TL;DR

User reported the host appeared to suspend / log out / hibernate during a long work session. **Systematic investigation found no evidence of any host-level power transition.** The host has been continuously up for 7h08m at the time of investigation, single user session unbroken since 15:42, all CONST-033 hardening intact, zero `will suspend` broadcasts since the 2026-04-26 fix.

The most likely actual experience: **GUI screen lock / display blank** (an X11/GNOME-shell layer behavior NOT subject to systemd power-management), OR a brief GUI freeze caused by a foreign podman container hitting its own 1 GB memory cgroup limit and OOM-killing its workers — which is exactly what the container’s own resource limit is designed to do (containment, not host impact).

No code we wrote contributed. The CONST-033 source-tree challenge passes; nothing in this project invokes any forbidden host-power call directly or indirectly.

Triple-check evidence

1. Source-tree clean (CONST-033 layer 5)

```
$ bash challenges/scripts/no_suspend_calls_challenge.sh
=== no_suspend_calls_challenge ===
Scanner: scripts/host-power-management/check-no-suspend-calls.sh
```

```
Root:    /run/media/milosvasic/DATA4TB/Projects/Boba
OK: no forbidden host-power-management calls
=== summary: PASS ===
```

2. Host hardened (CONST-033 layer 4)

```
$ bash challenges/scripts/host_no_auto_suspend_challenge.sh
[1/4] sleep / suspend / hibernate / hybrid-sleep targets masked?
PASS
[2/4] AllowSuspend=no in sleep.conf or drop-in?
PASS
[3/4] logind IdleAction safe? (IdleAction=ignore)
PASS
[4/4] 'will suspend' broadcasts since 2026-04-26 fix:    0
PASS
=== summary: 4 pass, 0 fail ===
```

3. No host-level suspend / shutdown / reboot occurred

```
$ uptime
 22:50:55 up  7:08,  1 user,  load average: 0.62, 0.87, 2.01
```

7h08m continuous uptime — no suspend, no resume, no reboot since the workday started. The reported “stuck or suspended” perception is incompatible with the host’s actual state.

4. No user session terminated

```
$ journalctl --user --since today | grep -iE "logout|terminated|
signed|session.*close"
# only su (root) sessions closing — milosvasic session unbroken
since 15:42 login
```

5. Memory state currently healthy

```
$ free -h
               total        used        free      shared  buff/
cache   available
Mem:            62Gi        11Gi       3.0Gi       4.7Gi
53Gi        51Gi
Swap:           15Gi       361Mi        15Gi

$ cat /proc/pressure/memory
some avg10=0.01 avg60=0.03 avg300=0.00 total=102743908
```

51 GiB available, ~zero memory pressure. Nothing approaching OOM territory at the host level.

6. The one OOM event found

```
$ journalctl -k --since "24 hours ago" | grep -iE "oom|killed
process"
Apr 27 21:41:35 nezha kernel: oom-
kill:constraint=CONSTRAINT_MEMCG, ...
  oom_memcg=/user.slice/user-1000.slice/user@1000.service/
user.slice/
            user-libpod_pod_41847d97.../libpod-d2fcb8aa.../scope
Apr 27 21:41:35 nezha kernel: Memory cgroup out of memory: Killed
process
  598707 (python3) total-vm:1108616kB anon-rss:70736kB ...
oom_score_adj:200
Apr 27 21:41:35 nezha kernel: memory: usage 1048568kB, limit
1048576kB
```

Decoded: - A **container** (libpod-d2fcb8aa…) inside a **pod** (libpod_pod_41847d97…) hit its own **1048576 kB = 1 GB cgroup memory limit**. - The kernel killed the container's `python3` worker (oom_score_adj=200, well above any host process). - This is **exactly the design intent of cgroup memory limits** — contain the blast radius to the container's own slice. Host memory remained healthy. - The hash `d2fcb8aa...` does not match any container we created in this Boba session. `podman inspect d2fcb8aa9aa7` reports "no such object" — the OOM-killed container has been GC'd. The pod hash `41847d97...` likewise doesn't appear in `podman pod ps -a`. - Most likely owner: a foreign project's container (the host runs many: 364 podman images, 25 containers across `helixgitpx_*`, `helixflow-*`, `proxy-*`, `lava-*`, `metube-*`, etc.) — NOT this Boba project.

7. Boba project's own podman footprint

```
$ podman images localhost/boba-jackett
REPOSITORY              TAG   IMAGE ID      CREATED      SIZE
localhost/boba-jackett  dev   51d69d8198a1  About 1h ago 22.8 MB

$ podman ps -a | grep -iE "boba|jackett"
jackett        lscr.io/linuxserver/jackett:latest    Exited
boba-jackett   localhost/boba_boba-jackett:latest    Exited
```

Both Boba containers are STOPPED. Our Dockerfile.jackett-built image is 22.8 MB (modernc/sqlite pure-Go, no CGO). Compose-defined limits (`mem_limit: 256m`, `pids_limit: 256`, `oom_score_adj: 500`) ensure that even if our container OOMs, it dies BEFORE other workloads.

Root cause: most likely

Three candidate explanations for the user's experience, ranked by evidence:

1. **GNOME / X11 screen lock or display blank.** Not a host suspend; a different layer entirely. CONST-033 covers systemd power targets but does NOT cover gsettings session locking. The session would appear "frozen" when really it just blanked the display. Coming back to it = "did it sleep?" Verifiable via `gsettings get org.gnome.desktop.session idle-delay` and `gsettings get org.gnome.desktop.screensaver lock-enabled` — out of scope for the project's CONST-033 ban (those settings don't change power state), but worth user-side audit for ergonomics.

2. **GNOME shell briefly unresponsive while the foreign podman container reached its OOM cap.** The `Mullvad VPN` "Frame has assigned frame counter but no frame drawn time" warnings at 22:33:13 in gnome-shell logs hint at compositor stalls around the same era. Brief stalls can present as "the screen froze for a moment".

3. **Some foreign automation hit a memory ceiling.** The OOM-killed container is from another project. No code in this Boba session can create a container in someone else's pod hash.

No evidence supports an actual host suspend, hibernate, or session logout.

Could podman / docker themselves cause host suspension?

Investigated: - **Rootless podman** (used here): runs entirely in user-mode. No systemd power-management interaction. The OOM-killer respects cgroup memory limits and kills containers, NOT the host or the user session. - **Docker daemon** (not used here): runs as root via systemd. Docker has never been observed to invoke power transitions; no upstream code path exists for it. - **systemd-oomd** (not enabled on this host — `systemd-oomd` user unit reports `No entries`): if enabled on a future host, it CAN kill entire cgroup slices including the user@1000 slice when system-wide pressure exceeds thresholds. That presents as a logout from the user's perspective. Mitigation: keep memory/pids limits on EVERY container, set `oom_score_adj` on long-running containers to be killed BEFORE the user session.

Verdict: podman on this host is safe by default. The cgroup OOM event proved containment worked (foreign container killed; host + user session unaffected).

Recurrence prevention (defense in depth)

1. **Container hygiene.** Every podman service in `docker-compose.yml` already has `mem_limit`, `pids_limit`, `oom_score_adj: 500` — confirmed present on the new `boba-jackett` service from Task 24. Continue this pattern for every new service.

2. **Resource limits on test runs (CONST-09).** Already enforced: `GOMAXPROCS=2 nice -n 19 ionice -c 3` on every `go test` invocation.

3. **Background process cleanup.** Always `kill` smoke-test binaries explicitly before claiming a task complete. Lingering background processes from prior smoke tests are themselves a memory-pressure source. Discovered one such leak this session (a Task-22 smoke binary left running on :7189) — cleaned up.

4. **Periodic podman GC.** Disk usage shows 176 GB in `~/.local/share/containers`, with 25 stopped containers and 39 GB reclaimable. A `podman system prune -a --volumes` should be a periodic user-level maintenance task. NOT something this project automates (cross-project blast radius), but worth flagging in CLAUDE.md.

5. **GUI screen-lock / display-blank policy** (user-environment tier, not project-tier): if the user wants no display blanking either, `gsettings set org.gnome.desktop.session idle-delay 0` and similar. Out of scope for this project's constitutional rules but worth a user-side toggle if the experience repeats.

Doctrine added (this commit)

The investigation findings are codified in: - `CONSTITUTION.md` § "CONST-033 Operational Note — Distinguishing Host Suspension from Adjacent Phenomena" - `CLAUDE.md` and `AGENTS.md` — short cross-references to the operational note + reminder to triple-check before assuming we caused a perceived power event.

Container clean-rebuild action taken

Per user request, after the investigation: - Stopped Boba's `boba-jackett` and `jackett` containers. - Removed our locally-built `localhost/boba-jackett:dev` image AND the compose-built `localhost/boba_boba-jackett:latest` image. - Will rebuild via `./`

`start.sh -p` cycle when the user starts the stack next (per the project's "no manual container commands" CONST rule — the orchestrator brings everything up).

Status

- No host suspend / hibernate / logout occurred.
- No code we shipped contributes any direct or indirect risk of host power transition.
- Both CONST-033 challenges PASS (source-tree + host-state).
- Foreign OOM event was contained at the cgroup boundary as designed.
- Doctrine codified for future incident triage.